

5 SQL schéma pro datové katalogy (specializace WDOP)

Navrhněte SQL schéma pro reprezentaci datových katalogů na základě DCAT slovníku. Schéma musí obsahovat reprezentaci pro následující entity a jejich vlastnosti:

- Entita datový katalog (`dcat:Catalog`) s vlastnostmi:
 - IRI
 - název (`dct:title`)
 - datové sady (`dcat:dataset`)
- Entita datová sada (`dcat:Dataset`) s vlastnostmi:
 - IRI
 - název (`dct:title`)
 - klíčová slova (`dct:keyword`)
 - distribuce (`dcat:distribution`)
- Entita distribuce datové sady (`dcat:Distribution`) s vlastnostmi:
 - IRI
 - URL pro stažení (`dcat:downloadURL`)
 - formát (`dcat:format`)
 - poslední úprava (`dct:modified`)

Při návrhu předpokládejte následující:

- název je pouze v jednom jazyce,
- datová sada může mít více klíčových slov,
- datová sada náleží do právě jednoho katalogu,
- distribuce náleží do právě jedné datové sady.

Vysvětlete 3. normální formu a zajistěte, aby jí navržené schéma splňovalo.

Napište SQL, které vytvoří uvedené tabulky a vhodná integritní omezení.

Nástin řešení Je potřeba 4 tabulek:

- Catalog
- Dataset
- Keyword
- Distribution

Splnění 3. NF je přímočaré.

SQL by mělo obsahovat:

- vytvoření tabulek (`CREATE TABLE`),
- použití vhodného typu pro `dct:modified`,
- definici umělých primárních klíčů (`PRIMARY KEY`), nebo využití IRI,
- (`CONSTRAINT`) cizích klíčů (`FOREIGN KEY`), včetně cascade delete (`ON DELETE CASCADE`).

6 JSON-schéma (specializace WDOP)

Navrhněte JSON schéma pro publikaci dat o datové sadě z databáze v předchozí otázce. Vlastnost „název“ (`dct:title`) bude povinná jak pro datovou sadu, tak pro katalog.

Nástin řešení Řešení je opět poměrně přímočaré, tedy pouze základ pro ilustraci, bez distribucí.

```
{
  "$schema": "https://json-schema.org/draft/2020-12/schema",
  "$id": "http://example.com/schema/data-catalog",
  "type": "object",
  "required": ["title"],
  "properties": {
    "title": { "type": "string" },
    "keywords": { "type": "array", "items": {"type": "string"}}
  }
}
```

7 JSON-LD (specializace WDOP)

Navrhněte JSON-LD kontext a popište úpravu JSON schématu tak, aby bylo možné na data pohlížet jako na RDF reprezentaci entity datové sady (`dcat:Dataset`) obsahující distribuce.

Nástin řešení JSON schéma rozšíříme o:

- Klíč `@id` pro IRI jednotlivých entit, je možné aliasovat.
- Klíč `@type` pro jednotlivé entity s odpovídající hodnotou, je možné aliasovat.
- Klíč `@context` pro kořen JSON dokumentu odkazující na soubor kontextu.

Je třeba také změnit reprezentaci `title` pro podporu indikace jazyka ve formátu JSON-LD language map, tj.

```
{
  "$schema": "https://json-schema.org/draft/2020-12/schema",
  "$id": "http://example.com/schema/data-catalog",
  "type": "object",
  "required": ["@context", "@id", "@type", "title"],
  "properties": {
```

```

    "@id": { "type" : "string" },
    "@type": { "const" : "Catalog" },
    "title": {
        "type" : "object",
        "required" : [ "en" ],
        "properties" : {
            "en": { "type" : "string" }
        }
    },
    "keywords": { "type" : "array", "items": {"type": "string"}}
}
}

```

Alternativně lze vložit kontext i to dokumentu přímo, což by ale vyžadovalo delší popis jeho vnitřku.

Vybraná část JSON-LD pro ilustraci - konkrétní IRI nemusí být správně, je ale třeba vědět, že tam nějaké odpovídající použitým slovníkům je:

```

{
  "@context" : {
    "@version": 1.1,
    "dcat": "http://www.w3.org/ns/dcat#",
    "dcterms": "http://purl.org/dc/terms/",
    "Catalog": "dcat:Catalog",
    "title": {
      "@id": "dcterms:title",
      "@container": "@language"
    },
    "keywords": "dcat:keyword"
  },
}

```

8 PHP (specializace WDOP)

Navrhněte kostru implementace PHP skriptu pro obsluhu HTTP GET dotazu pro získání JSON reprezentace datové sady. HTTP GET dotaz obsahuje v URL QUERY části název "title", který je použitý pro získání a vrácení první datové sady s daným názvem. Pokud je datová sada nalezena, PHP skript jí pošle jako odpověď na požadavek ve formátu JSON. Schéma JSON dokumentu není specifikováno, můžete využít schéma z předchozích otázek.

Obslužte vhodně chybové stavy pomocí HTTP stavových kódů.

Cílem není poskytnout úplnou implementaci, ale popsat její hlavní body.

Nástin řešení Základní záchytné body:

- Kontrola vstupního argumentu, třeba pomocí funkce `filter_input`.
- Připojení k SQL pomocí `mysqli`.
- Vytvoření a sanitizace query.
- Uzavření spojení do databáze.
- Ošetření pomocí 404 a 500.
- Zaslání `content-type` hlavičky pro JSON na klienta.
- Zaslání JSON reprezentace, klidně pomocí `json_encode`.